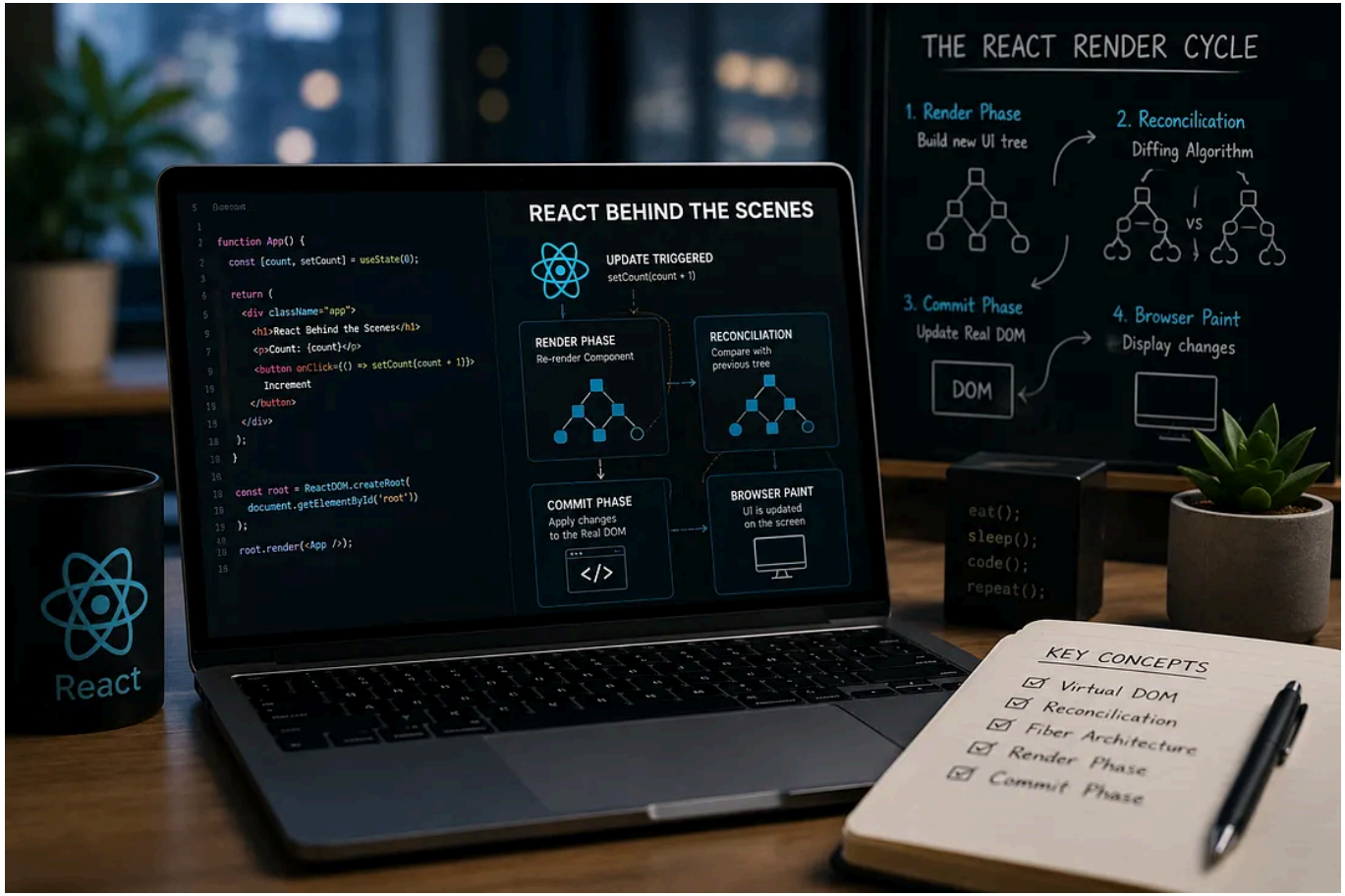
React Behind The Scenes: Virtual DOM, Reconciliation, Render Phase, Commit Phase & Fiber Explained.

Devendra Dhote Follow 5 min read · Jun 22, 2026

228 22 20



Most developers learn React through components, hooks, JSX, and state management. While these concepts are essential, very few developers truly understand what happens behind the scenes when React updates the UI.

In this article, we'll explore React's internal rendering process, understand the Virtual DOM, Reconciliation, Diffing Algorithm, Render Phase, Commit

Phase, and finally see how modern React uses Fiber Architecture to make updates more efficient.

Let's start from the beginning.

. . .

Why Was React Created?

Before React, developers primarily used JavaScript and jQuery to update the DOM manually.

```
const heading = document.querySelector("h1");
heading.textContent = "Hello World";
```

This approach works perfectly for small applications.

However, as applications became larger and more interactive, managing UI updates became increasingly difficult.

Imagine an application like Facebook:

- News Feed
- Notifications
- Messages
- Comments
- Likes
- Chat

Every section can update independently.

As the application grows, tracking which DOM node should update, when it should update, and how updates affect other parts of the UI becomes complex.

React solved this problem by introducing a new way of thinking:

> UI = Function(State)

Instead of manually updating the DOM, developers describe what the UI should look like, and React determines how to update the browser efficiently.

. . .

What Is The Real DOM?

When the browser parses HTML, it creates a tree-like structure known as the Document Object Model (DOM).

Example:

```
<body>
  <main>
    <h1>Hello</h1>
  </main>
</body>
```

DOM Tree:

```
Document
└── body
    └── main
        └── h1
            └── Hello
```

This structure is called the Real DOM because it is maintained directly by the browser.

Whenever changes occur in the Real DOM, the browser may need to recalculate layouts, repaint elements, and update the screen.

These operations can become expensive when applications grow larger.

. . .

What Is A React Element?

Consider the following code:

```
const element = React.createElement(
  "h1",
  {},
  "Hello React"
);
```

This does not create an actual DOM node.

Instead, React creates a plain JavaScript object:

```
{
  type: "h1",
  props: {
    children: "Hello React"
  }
}
```

This object is called a React Element.

A React Element is simply a description of what the UI should look like.

. . .

What Is The Virtual DOM?

The Virtual DOM is a JavaScript representation of the UI stored in memory.

For example:

```
<div>
  <h1>Hello</h1>
  <p>World</p>
</div>
```

can be represented internally as:

```
{
  type: "div",
  children: [
    {
      type: "h1",
      children: ["Hello"]
    },
    {
      type: "p",
      children: ["World"]
    }
  ]
}
```

The important thing to understand is:

> The Virtual DOM does not exist inside the browser.

It exists entirely in JavaScript memory.

. . .

Initial Render

When React renders for the first time:

```
const root = ReactDOM.createRoot(
  document.querySelector("main")
);
```

```
root.render(
  React.createElement(
    "h1",
    {},
    "Hello React"
  )
);
```

React performs the following steps:

```
React Element
      ↓
Internal React Tree
      ↓
Create Real DOM Nodes
      ↓
Attach To Browser DOM
```

At this point, the UI becomes visible on the screen.

. . .

What Happens When State Changes?

Let's assume the UI initially looks like this:

```
<h1>Hello</h1>
```

After a state update:

```
<h1>Hello React</h1>
```

React does not immediately update the Real DOM.

Instead, React starts a new rendering process.

. . .

Render Phase

The Render Phase is responsible for determining what changed.

During this phase React:

1. Re-executes the component.
2. Creates a new UI representation.
3. Compares it with the previous representation.
4. Determines the required updates.

Important:

> During the Render Phase, React does not touch the Real DOM.

Everything happens in memory.

. . .

Reconciliation

The process of comparing the previous UI tree with the newly generated UI tree is called Reconciliation.

Example:

Old Tree:

```
<h1>Hello</h1>
```

New Tree:

```
<h1>Hello React</h1>
```

React compares both trees and identifies what has changed.

. . .

Diffing Algorithm

Diffing is the algorithm React uses during Reconciliation.

Its job is to efficiently identify differences between the old tree and the new tree.

Example:

```
- Hello
+ Hello React
```

React now knows that only the text content changed.

It does not need to recreate the entire DOM node.

Important:

> React compares the previous Virtual DOM representation with the newly generated representation.

It does NOT compare the Real DOM with the Virtual DOM.

This is one of the most common misconceptions among React developers.

. . .

Render Phase Output

After Reconciliation and Diffing are completed, React prepares a list of changes.

Example:

```
Update Text Node
Remove Element
Insert Element
Update Props
```

At this stage, React knows exactly what needs to change.

However, the Real DOM has still not been updated.

. . .

Commit Phase

Once all changes have been calculated, React enters the Commit Phase.

During this phase React:

- Updates the Real DOM
- Updates refs
- Schedules effects
- Finalizes the UI update

Example:

```
h1.textContent = "Hello React";
```

The calculated updates are now applied to the browser's DOM.

. . .

Browser Paint

After the DOM is updated, the browser performs its own rendering work.

```
DOM Update
      ↓
```

```
Layout
      ↓
Paint
      ↓
User Sees Updated UI
```

Important:

> React updates the DOM.
>
> The browser performs the paint operation.

. . .

The Complete React Update Cycle

```
State Change
      ↓
Render Phase
      ↓
Create New UI Tree
      ↓
Reconciliation
      ↓
Diffing
```

```
      ↓
Prepare Updates
      ↓
Commit Phase
      ↓
Update Real DOM
      ↓
Browser Paint
```

. . .

Modern React: Enter Fiber

Everything we've discussed so far is conceptually correct.

However, modern React contains another important layer called Fiber.

Fiber was introduced in React 16 and completely rewrote React's rendering engine.

Many developers think:

```
Virtual DOM
      ↓
```

```
Real DOM
```

But modern React is closer to:

```
React Elements
      ↓
Fiber Tree
      ↓
Reconciliation
      ↓
Commit
      ↓
Real DOM
```

. . .

Why Was Fiber Introduced?

Before Fiber, React used synchronous rendering.

```
Start Work
      ↓
```

```
  Complete Work
```

Once React started rendering, it could not pause.

Large updates could block the browser and make the UI feel unresponsive.

Fiber solved this problem.

. . .

What Does Fiber Allow React To Do?

Fiber enables React to:

- Pause rendering work
- Resume rendering work
- Prioritize updates
- Interrupt low-priority tasks
- Keep the UI responsive

Example:

```
User Typing
      ↓
High Priority
```

```
Large List Rendering
      ↓
Low Priority
```

React can process the typing update first and continue rendering the list later.

. . .

Virtual DOM vs Fiber

A common question is:

“Is Fiber the Virtual DOM?”

Not exactly.

A simplified explanation is:

```
Virtual DOM
=
Conceptual UI Representation
```

```
Fiber
=
React's Internal Reconciliation Engine
```

When developers say “Virtual DOM,” they are usually referring to React’s internal UI representation.

Modern React uses Fiber Nodes and Fiber Trees internally to manage that representation efficiently.

. . .

Final Thoughts

React’s true strength is not simply the Virtual DOM.

Its real power comes from its ability to efficiently determine what changed, schedule updates intelligently, and update only the necessary parts of the UI.

Understanding concepts like:

- React Elements
- Virtual DOM
- Reconciliation
- Diffing Algorithm
- Render Phase
- Commit Phase
- Fiber Architecture

gives you a much deeper understanding of React than simply learning hooks and components.

The next time you call:

```
setState(...)
```

or

```
setCount(...)
```

you'll know exactly what happens behind the scenes before the UI updates on the screen.

React JavaScript Web Development Frontend Development React Core

Written by Devendra Dhote

242 followers · 8 following

MERN Stack Developer & Instructor. Writing about JavaScript, React, Next.js, backend development, and the engineering concepts behind modern web applications.

Follow